

Système intelligent de détection d'incendie basé sur l'IoT et l'intelligence artificielle

Réalisé par : Khaloufi Youssef, Oussama Allouch, Bourti Ayoub, ELKAIRAH Hassan, LOUAHABI Abdenour

Master IASD, S2 - Département Génie Informatique

Faculté des Sciences et Techniques de Tanger - Université Abdelmalek Essaâdi

Encadrant : Mohamed EL BRAK - Année universitaire : 2025/2026

Résumé

Ce travail présente la conception et la réalisation d'un prototype IoT intelligent pour la détection précoce d'incendie. Le système simule des mesures de température, fumée, gaz et humidité, publie les données au format JSON via MQTT, orchestre le traitement avec Node-RED, applique un modèle d'intelligence artificielle pour classer l'état de l'environnement et transmet les résultats vers ThingsBoard Cloud. L'objectif n'est pas uniquement d'afficher des capteurs, mais de construire une chaîne complète de bout en bout : acquisition edge, transport publish/subscribe, validation, enrichissement, décision automatique, alerte et visualisation. Le prototype est entièrement reproductible sans matériel physique : le script Python joue le rôle d'un Raspberry Pi simulé et l'actionneur est représenté par une alarme virtuelle publiée sur un topic MQTT. Les tests réalisés montrent le bon fonctionnement des trois états attendus, normal, suspect et danger, ainsi que la bonne intégration entre l'IA, Node-RED et le dashboard cloud.

Mots-clés

IoT, MQTT, Node-RED, ThingsBoard, intelligence artificielle, Random Forest, classification, incendie, capteurs simulés, dashboard, cloud.

1. Introduction

La surveillance des risques d'incendie constitue un cas d'usage pertinent pour illustrer l'intégration entre objets connectés, traitement de données en temps réel et intelligence artificielle. Dans un bâtiment réel, une augmentation rapide de la température, une concentration élevée de fumée ou de gaz et une baisse importante de l'humidité peuvent annoncer une situation dangereuse. Un système IoT doit alors collecter ces signaux, les transmettre rapidement, les analyser et déclencher une action compréhensible par l'utilisateur.

Le cahier des charges du module demande un système IoT complet de bout en bout. Le projet doit intégrer une couche objets ou edge, des capteurs et actionneurs réels ou simulés, une communication MQTT selon le modèle publish/subscribe, un format JSON, un traitement par Node-RED, une plateforme cloud IoT et une brique d'intelligence artificielle. Le travail présenté répond à ces exigences avec une architecture hybride : la génération des données est locale, l'orchestration est réalisée dans Node-RED, l'inférence IA est exécutée par un script Python et la visualisation finale est assurée par ThingsBoard Cloud.

Le choix d'un projet simulé permet de se concentrer sur l'architecture et sur la logique métier sans dépendre de composants électroniques. Le simulateur garde cependant une logique réaliste : il produit des scénarios cohérents, transmet des

messages JSON conformes et permet de tester les états normal, suspect et danger. Cette approche rend la démonstration plus stable, plus facilement reproductible et plus adaptée à une soutenance académique.

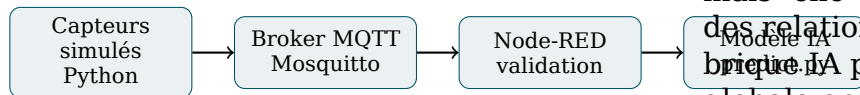


Figure 1 - Vue logique simplifiée de la chaîne IoT et IA.

La problématique retenue est volontairement claire : comment classer automatiquement l'état d'un environnement en trois classes, normal, suspect et danger, à partir de quatre mesures capteurs et comment transformer cette décision en alerte exploitable. La contribution principale du projet réside donc dans l'intégration cohérente de plusieurs technologies autour d'une démonstration unique.

2. Related work

L'Internet des Objets repose sur l'interconnexion de dispositifs capables de mesurer, transmettre et exploiter des données. Une architecture IoT typique contient une couche terrain, une couche de communication, une couche de traitement et une couche de stockage ou de visualisation. Dans notre projet, cette organisation est conservée même si la couche physique est simulée.

MQTT est un protocole léger de messagerie basé sur le modèle publish/subscribe. Il est largement utilisé dans les scénarios IoT parce qu'il réduit le couplage entre producteurs et consommateurs. Le simulateur publie ses messages sur un topic, tandis que Node-RED s'abonne à ce topic et traite les données reçues. Cette séparation permet de faire évoluer chaque composant sans modifier toute l'application.

Node-RED est une plateforme visuelle de programmation orientée flux. Elle facilite la construction d'applications IoT en reliant des nœuds de réception, transformation, décision et sortie. Dans notre cas, Node-RED reçoit les messages MQTT, vérifie leur structure, normalise les valeurs, applique une première logique de pré-alerte, appelle le modèle IA et publie l'alerte fi-

nale.

Les systèmes de détection d'incendie reposent souvent sur des seuils : si la température dépasse une valeur donnée ou si le niveau de fumée devient élevé, une alerte est déclenchée. Cette approche est simple, mais elle ne tient pas toujours compte des relations entre plusieurs mesures. Une brique IA permet de produire une décision globale en combinant température, fumée, gaz et humidité. Dans ce projet, l'IA ne remplace pas la logique IoT : elle l'enrichit.

ThingsBoard intervient comme plateforme cloud IoT. Son rôle est de recevoir les télémetries, de stocker les valeurs sous forme temporelle et de les afficher dans un tableau de bord. L'intérêt du cloud est de rendre les données accessibles pour la supervision, l'analyse et la présentation des résultats.

3. Solution proposée

La solution proposée est un pipeline séquentiel. Le simulateur Python génère des mesures capteurs et les publie via MQTT. Mosquitto joue le rôle de broker local. Node-RED reçoit les messages, les valide, les enrichit avec une première estimation puis appelle le script d'IA. Le script retourne un objet JSON contenant le statut IA, le score de risque et l'état de l'alerte. Node-RED fusionne cette sortie avec le message initial, publie l'alerte si nécessaire et transmet les valeurs enrichies vers ThingsBoard.

3.1 Objectifs fonctionnels

Le système doit satisfaire les objectifs suivants : générer automatiquement des données réalistes, publier les messages via MQTT, utiliser un format JSON commun, orchestrer le traitement dans Node-RED, classer l'état en normal, suspect ou danger, déclencher une alerte lorsque le danger est détecté et visualiser les résultats dans un dashboard cloud.

3.2 Répartition technique

Table 1 - Organisation technique du travail.

Membre	Responsabilité principale
Khaloufi Youssef	Simulation edge, format JSON et publication MQTT.
Oussama Al-louch	Flow Node-RED de base : réception, validation, normalisation et pre_alert.
Bourti Ayoub	Dataset IA, entraînement, évaluation et script de prédiction.
ELKAIRAH Hassan	Intégration IA dans Node-RED, parsing JSON, fusion des résultats et alerte MQTT.
LOUAHABI Abdenour	Cloud, dashboard ThingsBoard et preuve bout en bout.

3.3 Architecture générale

La chaîne réalisée sépare clairement les responsabilités. La couche edge produit les données. La communication MQTT transporte les messages. Node-RED joue le rôle d'orchestrateur. L'IA produit une classification. L'actionneur simulé représente l'alarme et ThingsBoard assure le stockage et la visualisation.

Listing 1 - Schéma textuel du pipeline réalisé.

```
Simulateur Python -> MQTT Mosquitto -> Node-RED ->
predict.py -> Alerte MQTT -> ThingsBoard
```

Cette séparation rend le projet plus facile à tester. Le simulateur peut être exécuté seul, le modèle IA peut être testé en ligne de commande, Node-RED peut afficher les messages intermédiaires et ThingBoard peut vérifier la réception des télé-métries.

4. Réalisation : couche objets et simulateur edge

La couche objets est représentée par un script Python. Ce script joue le rôle d'un Raspberry Pi simulé connecté à quatre capteurs : température, fumée, gaz et humidité. Les valeurs ne sont pas purement aléatoires. Elles sont générées selon des scénarios afin de conserver une cohérence entre les mesures et l'état attendu.

Le scénario normal correspond à une salle stable : température modérée, fumée faible, gaz faible et humidité correcte. Le scénario suspect représente une situation anormale, mais pas encore critique : certaines valeurs augmentent et l'humidité diminue légèrement. Le scénario danger combine plusieurs signaux forts : tempéra-

ture élevée, fumée élevée, gaz élevé et humidité faible.

```
[root@python ~]# python /usr/local/bin/fire-detection-ai.py sub-service monitor start
Enter authentication Password:
Connection to localhost:8080 [ssh://root@localhost:8080]:
Warning: Permanently added host key fingerprint to known hosts
Breaker MQTT : localhost:1883
Topic MQTT : IoT/FireSensor/data
Interval : 1 secondes
Connection to broker MQTT : localhost:1883
Message publish :
{"device": "fire_sensor_01", "timestamp": "2026-09-11T10:01:26", "temperature": 27.21, "smoke": 10, "gas": 66, "humidity": 49, "location": "hall1", "scenario": "normal"}
Connection MQTT requiesc.
Message publish :
{"device": "fire_sensor_01", "timestamp": "2026-09-11T10:01:26", "temperature": 26.9, "smoke": 171, "gas": 285, "humidity": 63, "location": "hall1", "scenario": "normal"}
Message publish :
{"device": "fire_sensor_01", "timestamp": "2026-09-11T10:01:27", "temperature": 21.69, "smoke": 70, "gas": 186, "humidity": 63, "location": "hall1", "scenario": "normal"}
Message publish :
```

Figure 2 - Exécution du simulateur Python et publication des messages capteurs.

Le simulateur peut produire des messages continus ou un seul message pour un test rapide. Cette flexibilité facilite la démonstration : pendant la phase de développement, un mode “once” permet de valider rapidement le format JSON ; pendant la soutenance, un mode continu permet de montrer l’évolution des mesures dans le dashboard.

4.1 Format JSON commun

Le format JSON est la base d'échange entre les composants. Tous les traitements reposent sur les mêmes noms de champs, ce qui évite les conversions inutiles et limite les erreurs d'intégration.

Listing 2 - Exemple de message capteur publié par le simulateur.

```
{
  "device_id": "fire_sensor_01",
  "timestamp": "2026-05-26T10:30:00",
  "temperature": 35.2,
  "smoke": 180,
  "gas": 220,
  "humidity": 45,
  "location": "Salle_1",
  "scenario": "normal"
}
```

Chaque champ a un rôle précis. `device_id` identifie la source, `timestamp` permet le suivi temporel, les quatre mesures servent à la classification, `location` indique l'emplacement et `scenario` facilite la vérification pendant les tests.

5. Communication MOTT

La communication locale repose sur Mosquitto, configuré sur localhost:1883. Le simulateur publie les messages capteurs sur le topic `iot/fire/sensor/data`. Node-RED s'abonne à ce topic pour recevoir les données en temps réel. L'alerte finale est publiée sur un topic séparé : `iot/fire/alert`.

Table 2 - Plages utilisées pour générer les classes du dataset.

Classe	Temp.	Fumée	Gaz	Humidité
normal	16-35°C	30-115	60-145	45-65%
suspect	41-60°C	230-510	250-450	31-47%
danger	63-100°C	700-1000	580-850	10-29%

Le dataset est volontairement simple afin de stabiliser la démonstration. Cette décision est cohérente avec l'objectif pédagogique : vérifier l'intégration IoT + IA plutôt que résoudre un problème industriel avec des données bruitées.

7.2 Choix du modèle

Le modèle retenu est un Random Forest Classifier. Ce choix est adapté à des données tabulaires de petite taille, car les forêts aléatoires gèrent bien les relations non linéaires et fournissent une importance des variables. Le modèle est aussi rapide à entraîner et simple à déployer dans un script Python.

Le script `train_model.py` charge le dataset, sépare les données en entraînement et test, entraîne le modèle, calcule les métriques, produit la matrice de confusion et sauvegarde le modèle sous forme `joblib`. Le script `predict.py` charge ensuite ce modèle pour prédire une classe à partir de nouvelles mesures.

```
C:\Users\USER\Desktop\iot-fire-detection-ai>iot-fire-detection-ai
i>python simulator\simulate_fire_sensors.py
Démarrage du simulateur IoT - Membre 1
Broker MQTT : localhost:1883
Topic MQTT : iot/fire/sensor/data
Intervalle : 3 secondes

-----
paho-mqtt n'est pas installé : passage en mode hors ligne.
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:04", "temperature": 27.97, "smoke": 47, "gas": 148, "humidity": 54, "location": "Salle_1", "scenario": "normal"}
-----
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:07", "temperature": 51.36, "smoke": 556, "gas": 392, "humidity": 36, "location": "Salle_1", "scenario": "suspect"}
-----
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:10", "temperature": 71.26, "smoke": 712, "gas": 647, "humidity": 15, "location": "Salle_1", "scenario": "danger"}
-----
Message généré (mode hors ligne) :
{"device_id": "fire_sensor_01", "timestamp": "2026-05-27T17:20:13", "temperature": 66.04, "smoke": 988, "gas": 771, "humidity": 21, "location": "Salle_1", "scenario": "danger"}
```

Figure 5 - Tests manuels de prédiction IA sur plusieurs scénarios.

8. Résultats du modèle IA

L'évaluation du modèle montre une accuracy de 100% sur le jeu de test. La validation croisée stratifiée donne également des scores parfaits. La matrice de confusion indique 10 prédictions correctes pour chaque classe et aucune confusion entre normal, suspect et danger.

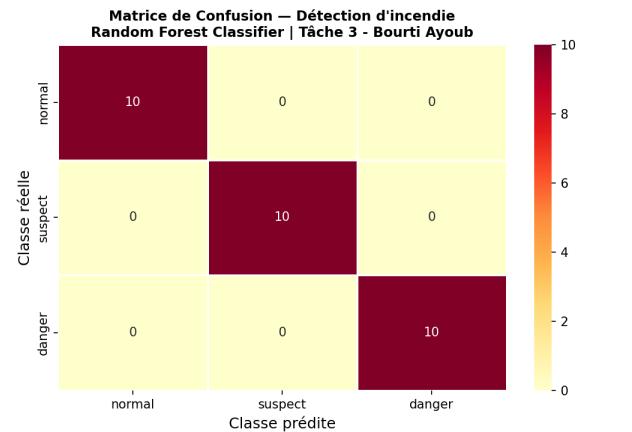


Figure 6 - Matrice de confusion du modèle Random Forest.

Ces résultats doivent être interprétés avec prudence. Ils confirment que le modèle apprend correctement les règles du dataset simulé et que l'intégration fonctionne. En revanche, ils ne prouvent pas une généralisation en environnement réel. Les classes sont bien séparées et les capteurs simulés ne produisent pas encore de bruit complexe, de dérive ou de panne.

Table 3 - Synthèse quantitative de l'évaluation IA.

Indicateur	Valeur	Interprétation
Échantillons	120	Dataset équilibré simulé.
Train/Test	90/30	Séparation 75% / 25%.
Classes	3	normal, suspect, danger.
Accuracy	100%	Aucune erreur sur le test contrôlé.
Variable forte	smoke	Importance élevée dans le modèle.

L'importance des variables indique que la fumée, la température et le gaz sont les signaux les plus déterminants. L'humidité joue un rôle complémentaire. Elle permet d'améliorer la séparation lorsqu'elle

accompagne des valeurs élevées de fumée et de gaz.

9. Intégration IA dans Node-RED

L'intégration IA constitue le point de jonction entre la modélisation et l'architecture IoT. Node-RED prépare les quatre valeurs numériques, construit l'appel vers le script Python, récupère la sortie JSON et ajoute les champs IA au message courant. Cette solution est simple, lisible et suffisante pour la démonstration.

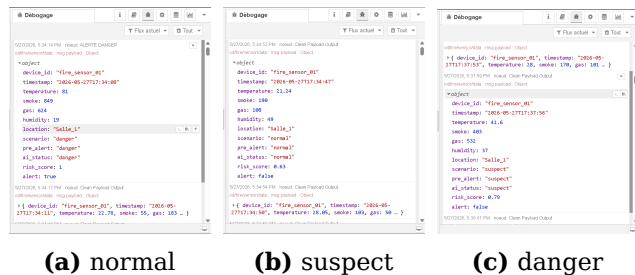


Figure 7 - Messages enrichis dans le debug Node-RED pour les trois scénarios.

Le cas normal produit `ai_status=normal` et `alert=false`. Le cas suspect produit `ai_status=suspect` mais ne déclenche pas l'alerte finale. Le cas danger produit `ai_status=danger` et `alert=true`. Cette différence est importante : tous les comportements anormaux ne doivent pas déclencher une alarme critique.

9.1 Différence entre `pre_alert`, `ai_status` et `alert`

`pre_alert` correspond à une estimation rapide basée sur des règles dans Node-RED. `ai_status` correspond à la classification finale produite par le modèle. `alert` est le booléen d'action : il vaut `true` uniquement lorsque le statut final est danger. Cette séparation évite de confondre observation, décision et action.

10. Cloud et dashboard ThingsBoard

La plateforme cloud utilisée est ThingsBoard Cloud. Les données enrichies sont envoyées via MQTT vers le topic de télémétrie standard. ThingsBoard joue deux rôles : stockage temporel des mesures et visualisation dans un dashboard.

Le tableau de bord affiche les courbes de température, fumée, gaz et humidité. Il affiche aussi le statut IA et l'état de l'alerte. Cette visualisation permet de vérifier que les trois scénarios produisent des comportements distincts et que la décision IA est bien transmise jusqu'au cloud.

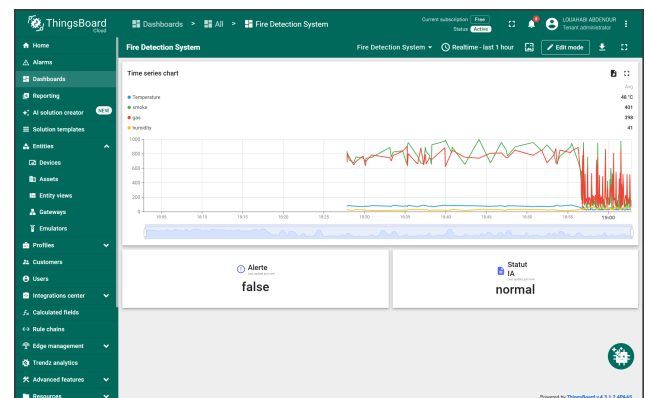
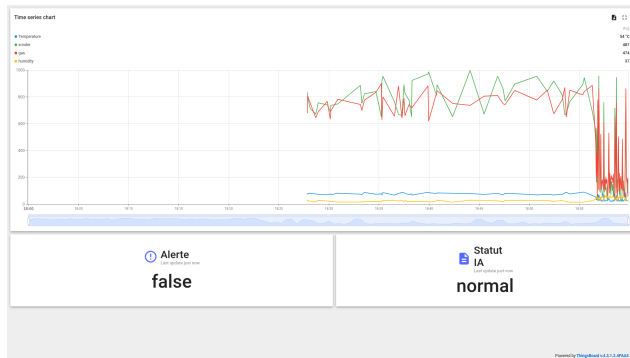


Figure 8 - Dashboard ThingsBoard complet avec télémétrie et statut IA.

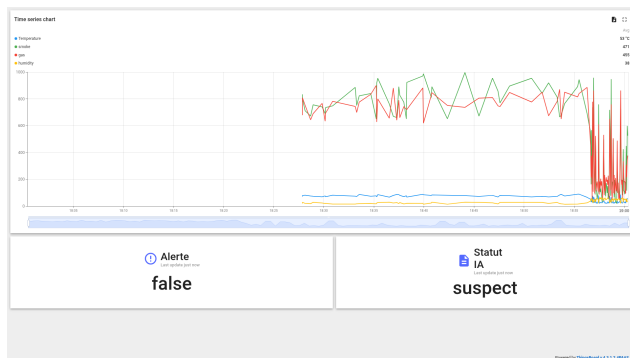
La configuration cloud repose sur un token de device. Ce token doit rester privé et ne doit pas être publié dans GitHub. Le dépôt contient donc des exemples anonymisés ou des placeholders. Cette décision garde la démonstration reproductible tout en respectant les bonnes pratiques de sécurité.

10.1 Données envoyées au dashboard

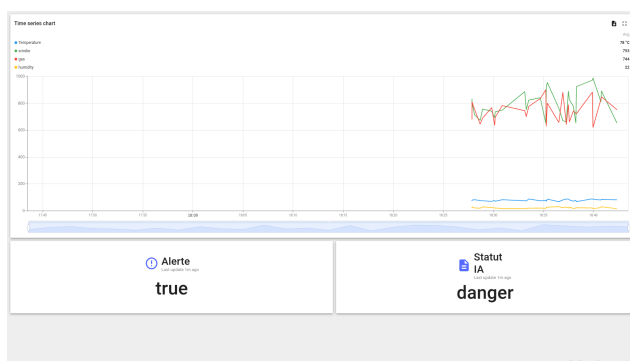
Les champs transmis au cloud incluent les mesures physiques, l'emplacement, le scénario simulé, la pré-alerte, le statut IA, le score de risque et l'état final de l'alerte. Le dashboard ne montre donc pas seulement les capteurs : il montre aussi la décision intelligente produite par le système.



(a) Scénario normal : alert=false.



(b) Scénario suspect : alert=false.



(c) Scénario danger : alert=true.

Figure 9 - Validation visuelle des trois états dans ThingsBoard.

11. Démonstration bout en bout

La validation complète suit le chemin réel des données. Le simulateur génère une mesure, Mosquitto transporte le message, Node-RED le reçoit, l'IA le classe, l'alerte est déclenchée si nécessaire, puis les valeurs enrichies sont visualisées dans ThingsBoard. Cette démonstration confirme que les composants ne sont pas isolés, mais reliés dans une chaîne fonctionnelle.

11.1 Scénario normal

Dans le scénario normal, les valeurs restent dans des plages stables. Node-RED re-

çoit les mesures, le modèle retourne le statut normal, le score de risque reste faible et alert=false. Aucune publication critique n'est envoyée sur le topic d'alerte.

11.2 Scénario suspect

Dans le scénario suspect, certaines valeurs augmentent, mais la décision finale reste non critique. Le statut suspect permet d'attirer l'attention sans déclencher l'alarme. Ce niveau intermédiaire est utile pour éviter les alertes excessives.

11.3 Scénario danger

Dans le scénario danger, plusieurs signaux sont forts simultanément. Le modèle retourne danger, le score de risque est élevé et alert=true. Node-RED publie alors l'alerte sur `iot/fire/alert`. Dans une version matérielle, cette étape pourrait activer un buzzer, une LED ou un relais.

Listing 5 - Ordre conseillé pour la démonstration en soutenance.

```
sudo service mosquitto start
node-red
# importer et deployer le flow
python simulator/simulate_fire_sensors.py
mosquitto_sub -h localhost -t iot/fire/sensor/data
mosquitto_sub -h localhost -t iot/fire/alert
python ai/predict.py 78 850 720 22
```

12. Analyse critique

Les résultats obtenus valident le fonctionnement global du prototype. La chaîne couvre les principales couches demandées : objets simulés, communication MQTT, orchestration Node-RED, logique de décision, IA, actionneur simulé et visualisation cloud. Le projet est également bien organisé, car chaque dossier du dépôt correspond à une responsabilité technique.

La première limite concerne les données. Le dataset est simulé et les classes sont bien séparées. Cela explique les scores très élevés du modèle. Dans un environnement réel, les capteurs peuvent produire du bruit, dériver avec le temps, tomber en panne ou mesurer des situations ambiguës. Une version avancée devrait donc intégrer des données réelles et des cas limites.

La deuxième limite concerne l'intégration IA par appel de script. Cette méthode est pratique et lisible, mais elle peut devenir fragile dans un système plus grand. Une API REST locale ou un microservice conteneurisé rendrait l'inférence plus mainte-

nable et faciliterait la gestion des erreurs.

La troisième limite concerne la sécurité. Le prototype local ne traite pas encore l'authentification MQTT avancée, le chiffrement TLS, la rotation des tokens cloud ou le contrôle d'accès utilisateur. Ces aspects seraient nécessaires pour un déploiement réel.

13. Structure du dépôt

Le dépôt est organisé pour séparer les responsabilités. Le dossier `simulator` contient la couche edge, `ai` contient le dataset, le modèle et les scripts de prédiction, `flow` contient le flow Node-RED de base, `integration_ia_node_red` contient l'intégration IA, `cloud` documente ThingsBoard, et `screenshots` regroupe les preuves visuelles.

```
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$ find . -maxdepth 2 -type f | sort
./gitignore
./PROJECT_STATUS.md
./README.md
./ai/confusion_matrix.png
./ai/dataset_fire_detection.csv
./ai/evaluation_results.txt
./ai/model_fire_detection.joblib
./ai/predict.py
./ai/train_model.py
./article_screenshots/01_simulator_mqtt_publish.png
./article_screenshots/02_mqtt_subscriber_received_json.png
./article_screenshots/03_node_red_final_flow.png
./article_screenshots/04_node_red_debug_normal.png
./article_screenshots/05_node_red_debug_suspect.png
./article_screenshots/06_node_red_debug_danger_alert.png
./article_screenshots/07_ai_prediction_tests.png
./article_screenshots/08_confusion_matrix.png
./article_screenshots/09_evaluation_results.txt
./article_screenshots/dashboard_danger.png
./article_screenshots/dashboard_normal.png
./article_screenshots/dashboard_suspect.png
./article_screenshots/thingsboard_dashboard.png
./cloud/README_demo.md
./cloud/preuve_test_bout_en_bout.md
./config.example.json
./docs/README_node_red_base(membre2).md
./docs/README_task3.md
./docs/checklist_soutenance.md
./docs/mqtt_format_membre1.md
./docs/validation_membre1.md
./Flow/flow_node_red_base.json
./integration_ia_node_red/README_integration_ia.md
./integration_ia_node_red/flow_node_red_ai_alert.json
./iot-fire-detection-ai.pdf
./logs/sample_output.txt
./requirements.txt
./screenshots/capture_node_red_reception.png
./screenshots/danger_alert.png
./screenshots/dashboard_danger.png
./screenshots/dashboard_normal.png
./screenshots/dashboard_suspect.png
./screenshots/normal_alert.png
./screenshots/suspect_alert.png
./screenshots/thingsboard_dashboard.png
./simulator/config.json
./simulator/simulate_fire_sensors.py
./tests/mqtt_subscriber_test.py
(youssef) youssef@youssef:/mnt/e/projects/iot-fire-detection-ai$
```

Figure 10 - Structure technique du dépôt du projet.

Cette organisation rend le projet plus lisible pour l'évaluation. Elle permet aussi de relancer uniquement une partie de la chaîne : tester le simulateur, réentraîner le modèle, importer le flow ou vérifier le dashboard.

14. Perspectives

Plusieurs améliorations peuvent être envisagées. La première consiste à remplacer les capteurs simulés par des capteurs réels,

par exemple un capteur DHT pour la température et l'humidité, un capteur MQ pour le gaz et un capteur de fumée. Le script Python pourrait alors être exécuté sur un Raspberry Pi réel.

Une deuxième amélioration serait d'ajouter un actionneur physique, comme un buzzer, une LED ou un relais. L'actionneur serait commandé lorsque `alert=true`. Cette extension permettrait de transformer l'alarme virtuelle en action visible dans l'environnement.

Une troisième amélioration concerne l'IA. Le dataset pourrait être enrichi avec des valeurs bruitées, des capteurs défectueux, des scénarios progressifs et des cas ambigus. Il serait aussi possible de comparer plusieurs modèles : Random Forest, SVM, KNN, XGBoost ou réseau de neurones léger. Une détection d'anomalies non supervisée pourrait compléter la classification.

Enfin, le système pourrait intégrer des notifications Telegram ou email, un historique long terme, une authentification MQTT sécurisée, une connexion TLS et un déploiement conteneurisé. Ces améliorations rapprocheraient le prototype d'un système opérationnel.

15. Méthodologie de validation

La validation du projet n'a pas été limitée à un seul test final. Elle a été organisée par couches afin d'identifier rapidement l'origine d'une erreur. Cette méthode est importante dans un projet IoT, car un dysfonctionnement peut venir du simulateur, du broker MQTT, du flow Node-RED, du script IA, du parsing JSON ou de la plateforme cloud.

La première étape a consisté à vérifier la génération locale des messages. Le simulateur devait produire des valeurs cohérentes avec le scénario choisi et respecter le format JSON attendu. Ensuite, la réception MQTT a été validée avec un subscriber indépendant. Cette vérification permet de confirmer que le problème ne vient pas du transport lorsque Node-RED ne reçoit rien.

La deuxième étape a porté sur Node-RED. Chaque nœud important a été testé avec le panneau debug : payload reçu,

payload normalisé, pré-alerte, commande IA, sortie du script, parsing JSON et message final enrichi. Cette approche rend le flow plus facile à expliquer pendant la soutenance, car chaque transformation peut être observée.

La troisième étape a concerné l'IA. Le modèle a été testé séparément en ligne de commande avec des exemples normal, suspect et danger. Cette séparation évite de mélanger les erreurs de prédiction avec les erreurs d'intégration. Une fois ces tests validés, le script a été appelé depuis Node-RED.

Table 4 - Plan de validation utilisé pour la démonstration.

Couche	Test	Résultat attendu
Simulateur	Message JSON local	Champs complets et valeurs cohérentes.
MQTT	Subscriber Mosquitto	Réception sur <code>iot/fire/sensor/data</code> .
Node-RED	Debug après validation	Valeurs numériques normalisées.
IA	<code>predict.py</code> en CLI	Classe normal, suspect ou danger.
Alerte	Topic <code>iot/fire/alert</code>	Publication si danger uniquement.
Cloud	Dashboard ThingsBoard	Télémetrie et statut IA visibles.

Cette méthodologie montre que la validation est progressive. Elle donne aussi une stratégie claire pour corriger le projet en cas de panne : tester d'abord la génération, puis le transport, puis l'orchestration, puis l'IA et enfin le cloud.

16. Difficultés rencontrées

Plusieurs difficultés techniques sont apparues pendant la réalisation. La première concerne la cohérence des topics MQTT. Si le simulateur publie sur un topic différent de celui écouté par Node-RED, aucune donnée n'arrive dans le flow. La solution consiste à centraliser les noms de topics dans la configuration et à les rappeler dans la documentation.

La deuxième difficulté concerne le format JSON. Node-RED peut recevoir un payload sous forme de texte ou d'objet selon la configuration du nœud. Il faut donc convertir explicitement le message lorsque nécessaire et contrôler la présence des champs obligatoires avant de lancer la suite du traitement.

tement.

La troisième difficulté concerne l'appel du script IA. Le chemin de `predict.py` doit être correct dans le nœud `exec`. Si Node-RED est lancé depuis un autre répertoire, un chemin relatif peut échouer. La solution est de documenter le chemin attendu ou d'utiliser un chemin absolu lors de la démonstration.

La quatrième difficulté concerne ThingsBoard. Le token du device est obligatoire pour publier les données. Un mauvais token donne une impression de panne alors que le flow local fonctionne. Pour éviter ce problème, le dépôt conserve un fichier d'exemple sans token réel et la documentation précise les champs à remplacer.

17. Traçabilité des livrables

Le projet a été organisé pour que chaque livrable puisse être retrouvé facilement. Les scripts Python sont séparés des flows Node-RED, les captures de validation sont regroupées, et la partie cloud est documentée dans un dossier dédié. Cette traçabilité est importante parce qu'un rapport d'article ne suffit pas : l'évaluation doit pouvoir relier les affirmations du rapport aux fichiers techniques.

Table 5 - Correspondance entre exigences et livrables du dépôt.

Exigence	Livrable correspondant
Capteurs simulés	Script de génération edge et scénarios normal, suspect, danger.
Configuration MQTT	Fichiers de configuration locale et exemple sans token privé.
Flow Node-RED	Flow de réception, validation, normalisation et pré-alerte.
Modèle IA	Scripts d'entraînement, prédiction et modèle sauvegardé.
Dataset	Fichier CSV équilibré contenant les mesures et labels.
Intégration IA	Flow Node-RED avec appel Python, parsing JSON et alerte.
Cloud	Guide ThingsBoard, topic télémetrie et captures dashboard.
Preuves	Captures de simulation, MQTT, Node-RED, IA et cloud.

Cette organisation facilite aussi le travail d'équipe. Chaque membre peut présenter sa partie sans mélanger les fichiers, puis le chef de groupe peut relier les contributions dans une démonstration globale.

18. Synthèse pédagogique

Le projet illustre plusieurs notions vues dans le module. Le modèle publish/subscribe est montré par la séparation entre simulateur, broker et consommateurs. Le format JSON est utilisé comme contrat commun. Node-RED montre l'orchestration par flux, avec des étapes de validation, transformation et décision. ThingsBoard montre la partie cloud, stockage et visualisation. Enfin, la brique IA introduit une décision apprise à partir de données.

L'intérêt pédagogique est donc double. D'un côté, le système reste assez simple pour être expliqué clairement. De l'autre, il contient suffisamment de composants pour représenter une vraie architecture IoT. Cette combinaison rend le projet adapté à une soutenance : l'enseignant peut vérifier chaque couche, et l'équipe peut montrer les preuves techniques dans un ordre logique.

19. Conclusion

Ce projet a permis de concevoir un prototype complet de détection intelligente d'incendie basé sur l'IoT et l'intelligence artificielle. La chaîne réalisée couvre la génération des données capteurs, la publication MQTT, l'orchestration Node-RED, la classification IA, l'actionneur simulé d'alerte, le stockage cloud et la visualisation sur ThingsBoard.

Le système répond aux objectifs pédagogiques du module, car il combine les concepts IoT étudiés en cours avec une brique IA autonome. Même si le prototype reste simulé, il constitue une base claire pour une extension matérielle et pour une architecture plus robuste de supervision incendie.

La contribution principale est donc l'intégration cohérente de plusieurs technologies autour d'un cas d'usage compréhensible. Le modèle IA ne remplace pas la logique IoT, mais l'enrichit en produisant une décision structurée. L'ensemble forme une démonstration reproductible, lisible et adaptée à une soutenance de projet de fin de module.

Références

- [1] MQTT.org, *MQTT : The Standard for IoT Messaging*, <https://mqtt.org/>.
- [2] OASIS, *MQTT Version 5.0 OASIS Standard*, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [3] Node-RED, *Documentation officielle*, <https://nodered.org/docs/>.
- [4] Node-RED Cookbook, *Connect to an MQTT Broker*, <https://cookbook.nodered.org/mqtt/connect-to-broker>.
- [5] ThingsBoard, *MQTT Telemetry API*, <https://thingsboard.io/docs/pe/reference/mqtt-api/telemetry/>.
- [6] ThingsBoard, *Open-source IoT Platform*, <https://thingsboard.io/>.
- [7] Scikit-learn, *RandomForestClassifier documentation*, <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>.
- [8] Dépôt GitHub du projet : <https://github.com/KYoussefAI/iot-fire-detection-ai>.
- [9] Cahier des charges du module IoT, Pr. M. EL BRAK, FST Tanger, 2025/2026.

Annexe : commandes utiles

```
# Installation Python
pip install -r requirements.txt

# Lancer Mosquitto
sudo service mosquitto start

# Lancer Node-RED
node-red

# Lancer le simulateur
python simulator/simulate_fire_sensors.py
python simulator/simulate_fire_sensors.py --once
python simulator/simulate_fire_sensors.py --offline --once

# Tester le modele IA
python ai/predict.py 24 80 120 55
python ai/predict.py 48 340 300 40
python ai/predict.py 78 850 720 22
```

Annexe : lecture des sorties

Le système produit trois types de sorties. Les sorties MQTT brutes permettent de confirmer que les capteurs simulés publient correctement. Les sorties Node-RED permettent de vérifier la validation, la normalisation, l'appel IA et la fusion des champs. Les sorties ThingsBoard permettent de confirmer que les données enrichies arrivent dans le cloud et sont affichées dans le dashboard. Cette lecture en

trois niveaux facilite le diagnostic : transport, traitement et visualisation.

Annexe : checklist de soutenance

Pour présenter le projet de manière claire, la démonstration peut suivre une progression simple. D'abord, afficher la structure du dépôt pour rappeler les responsabilités. Ensuite, lancer Mosquitto et Node-RED. Après cela, exécuter le simulateur et montrer la réception MQTT. Une fois le transport validé, afficher le debug Node-RED pour expliquer la normalisation, la pré-alerte et l'appel au script IA. Enfin, déclencher un scénario danger et montrer l'alerte sur le dashboard ThingsBoard.

1. Présenter le problème : détection précoce d'incendie à partir de capteurs.
2. Expliquer l'architecture : simulateur, MQTT, Node-RED, IA, alerte et cloud.
3. Montrer un message JSON brut puis un message enrichi par l'IA.
4. Comparer les scénarios normal, suspect et danger.
5. Interpréter la matrice de confusion et rappeler la limite du dataset simulé.
6. Conclure avec les perspectives matérielles : Raspberry Pi, capteurs réels et buzzer.

Annexe : glossaire rapide

Edge : couche proche des capteurs, représentée ici par le simulateur Python. **Broker MQTT** : intermédiaire qui reçoit les publications et les distribue aux abonnés. **Topic** : canal logique MQTT, par exemple `iot/fire/sensor/data`. **Node-RED** : outil d'orchestration visuelle par flux. **Télémétrie** : données envoyées vers la plateforme cloud pour stockage et visualisation. **Actionneur simulé** : action logique représentant l'alarme, matérialisée par `alert=true` et par la publication sur `iot/fire/alert`.

Annexe : contrôles finaux

Avant de rendre le projet, trois contrôles sont nécessaires. Le premier est fonctionnel : les trois scénarios doivent produire les trois statuts attendus. Le deuxième est documentaire : les captures doivent cor-

respondre aux fichiers réellement présents dans le dépôt. Le troisième est sécurité : aucun token ThingsBoard réel, mot de passe ou clé privée ne doit être visible dans GitHub ou dans le rapport.

Table 6 - Points à vérifier avant la livraison.

Point	Vérification
Topics	Les noms MQTT sont identiques dans le simulateur, Node-RED et la documentation.
JSON	Les champs obligatoires sont présents et convertis en valeurs numériques.
IA	Les tests CLI retournent normal, suspect et danger selon les entrées.
Alerte	Le topic d'alerte reçoit un message uniquement pour le danger.
Cloud	Le dashboard affiche les mesures et le statut IA.
Sécurité	Les tokens réels sont remplacés par des placeholders.

Ces contrôles permettent de rendre un projet cohérent : le rapport, le code, les flows et les preuves visuelles racontent la même chaîne technique.